

STUDENT REFERENCE NOTES

Converting an HTML/CSS Grid-Based Web Page into React Functional Components

Implementing Login and Signup Forms Using Props and State

1. Learning Objectives

- Convert an HTML/CSS page into React JSX.
- Create React functional components.
- Pass data from a parent component using props.
- Manage changing form data using `useState()`.
- Handle input changes and form submission.
- Create Login and Signup forms.
- Use conditional rendering to switch between Login and Signup.

2. React Concepts Used

Concept	Purpose	Example
Component	Reusable UI building block	<code>function Login()</code>
JSX	HTML-like syntax inside JavaScript	<code><h1>Hello</h1></code>
Props	Pass data from parent to child	<code><Login title="Student Login" /></code>
State	Store data that can change	<code>const [email, setEmail] = useState("")</code>
<code>onChange</code>	Respond to input changes	<code>onChange={...}</code>
<code>onSubmit</code>	Respond to form submission	<code><form onSubmit={handleLogin}></code>
Conditional Rendering	Display UI based on state	<code>{showLogin ? <Login /> : <Signup />}</code>

3. Create the React Project

Open VS Code and run the following commands in the terminal.

```
npm create vite@latest react-auth-app
```

```
cd react-auth-app
```

```
npm install
```

```
npm run dev
```

Select React and JavaScript when Vite asks for the framework and variant.

4. Project Structure

```
react-auth-app
├── src
│   ├── App.jsx
│   ├── Header.jsx
│   ├── Sidebar.jsx
│   ├── MainContent.jsx
│   ├── Footer.jsx
│   ├── Login.jsx
│   └── Signup.jsx
```

```
├─ main.jsx
└─ index.css
```

5. Create the CSS Grid Layout

Open src/index.css and add:

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
}

.page {
  min-height: 100vh;
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: 100px 1fr 50px;
}

header {
  grid-column: 1 / 3;
  padding: 10px;
  background: #3888dd;
  color: white;
}

aside {
  padding: 20px;
  background: #eee;
}

main {
  padding: 20px;
}

footer {
  grid-column: 1 / 3;
  padding: 15px;
  text-align: center;
  background: #3888dd;
  color: white;
}

form {
  width: 300px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

input, button {
  padding: 10px;
}
```

```
1 +-----+
2 |           Header           |
3 +-----+-----+
4 | Sidebar | Content |
5 +-----+-----+
```

```
+-----+
|           Header           |
+-----+-----+
| Sidebar | Content |
+-----+-----+
|           Footer           |
+-----+-----+
```

6. Create the Header Component

Create src/Header.jsx:

```
function Header() {
  return (
    <header>
      <h1>My Website</h1>
    </header>
  );
}

export default Header;
```

7. Create the Sidebar Component

Create src/Sidebar.jsx:

```
function Sidebar() {
  return (
    <aside>
      <h3>Menu</h3>
      <p>Home</p>
      <p>Products</p>
      <p>Login</p>
    </aside>
  );
}

export default Sidebar;
```

8. Create MainContent and Footer

Create src/MainContent.jsx:

```
function MainContent() {
  return (
    <main>
      <h2>Welcome</h2>
      <p>This is the main content.</p>
    </main>
  );
}

export default MainContent;
```

Create src/Footer.jsx:

```
function Footer() {
  return (
    <footer>
      <p>© 2026 My Website</p>
    </footer>
  );
}

export default Footer;
```

9. Use Components in App.jsx

```
import Header from "../Header";
import Sidebar from "../Sidebar";
import MainContent from "../MainContent";
import Footer from "../Footer";

function App() {
  return (
    <div className="page">
      <Header />
      <Sidebar />
      <MainContent />
      <Footer />
    </div>
  );
}

export default App;
```

10. Props – Passing Data to a Component

Props are values passed from a parent component to a child component.

Change the Header call in App.jsx:

```
<Header title="KLECommerce" user="Student" />
```

Update Header.jsx:

```
function Header({ title, user }) {
  return (
    <header>
      <h2>{title}</h2>
      <p>Welcome, {user}</p>
    </header>
  );
}

export default Header;
```

Data flow: App → props → Header → displayed UI.

State (useState)

- **Concept:** State holds dynamic memory inside a component. When state changes, React automatically re-renders the component to show the updated value.
- **Tiny Example:**

```
const [count, setCount] = useState(0);  
// count = current value (0)  
// setCount = function used to update the value
```
- **Hands-On Use:** We will create src/Login.jsx using useState to store what the user types.

Controlled Inputs (onChange)

- **Concept:** In React, we keep input fields synchronized with state. Whenever the user types, the onChange event updates state, and the value attribute reads from state.
- **Tiny Example:**

```
<input  
  value={text}  
  onChange={(e) => setText(e.target.value)}  
>
```

11. Create the Login Form

Create src/Login.jsx:

```
import { useState } from "react";  
  
function Login({ title }) {  
  
  const [email, setEmail] = useState("");  
  const [password, setPassword] = useState("");  
  
  function handleLogin(e) {  
    e.preventDefault();  
  
    console.log("Email:", email);  
    console.log("Password:", password);  
  }  
  
  return (  
    <div>  
      <h2>{title}</h2>  
  
      <form onSubmit={handleLogin}>  
        <input  
          type="email"  
          placeholder="Enter Email"  
          value={email}  
          onChange={(e) => setEmail(e.target.value)}  
        />  
  
        <input
```

```

        type="password"
        placeholder="Enter Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />

      <button type="submit">Login</button>
    </form>
  </div>
);
}

export default Login;

```

12. Understand useState()

The statement below creates a state variable named email and a function named setEmail.

```
const [email, setEmail] = useState("");
```

Part	Meaning	Example
Email	Current state value	email
setEmail	Updates the state	setEmail(newValue)
""	Initial value	""

13. Connect an Input to State

```

<input
  type="email"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>

```

The input flow is: User types → onChange → setEmail() → state changes → React updates the UI.

14. Handle Form Submission

Use onSubmit to call the login function.

```

function handleLogin(e) {
  e.preventDefault();

  console.log("Email:", email);
  console.log("Password:", password);
}

```

e.preventDefault() stops the browser from performing its normal form submission and page reload.

15. Create the Signup Form

Create src/Signup.jsx:

```

import { useState } from "react";

function Signup() {

```

```

const [name, setName] = useState("");
const [email, setEmail] = useState("");
const [password, setPassword] = useState("");

function handleSignup(e) {
  e.preventDefault();

  console.log("Name:", name);
  console.log("Email:", email);
  console.log("Password:", password);
}

return (
  <div>
    <h2>Signup</h2>

    <form onSubmit={handleSignup}>
      <input
        type="text"
        placeholder="Enter Name"
        value={name}
        onChange={(e) => setName(e.target.value)}
      />

      <input
        type="email"
        placeholder="Enter Email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
      />

      <input
        type="password"
        placeholder="Enter Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />

      <button type="submit">Signup</button>
    </form>
  </div>
);
}

export default Signup;

```

16. Display Login and Signup in App

Import both components in App.jsx:

```

import Login from "../Login";
import Signup from "../Signup";

```

Place them inside the main content area:

```

<main>
  <h2>Authentication</h2>

```

```

    <Login title="Student Login" />
    <Signup />
  </main>

```

17. Switch Between Login and Signup

Import useState into App.jsx:

```
import { useState } from "react";
```

Create a state variable:

```
const [showLogin, setShowLogin] = useState(true);
```

Add buttons:

```

<button onClick={() => setShowLogin(true)}>
  Login
</button>

```

```

<button onClick={() => setShowLogin(false)}>
  Signup
</button>

```

Display one component based on state:

```
{showLogin ? <Login title="Student Login" /> : <Signup />}
```

18. Final App.jsx

```

import { useState } from "react";
import Header from "../Header";
import Sidebar from "../Sidebar";
import Login from "../Login";
import Signup from "../Signup";
import Footer from "../Footer";

```

```

function App() {
  const [showLogin, setShowLogin] = useState(true);

  return (
    <div className="page">

      <Header title="KLECommerce" user="Student" />

      <Sidebar />

      <main>
        <h2>Authentication</h2>

        <button onClick={() => setShowLogin(true)}>
          Login
        </button>

        <button onClick={() => setShowLogin(false)}>
          Signup
        </button>
      </main>
    </div>
  );
}

```

```

        <hr />

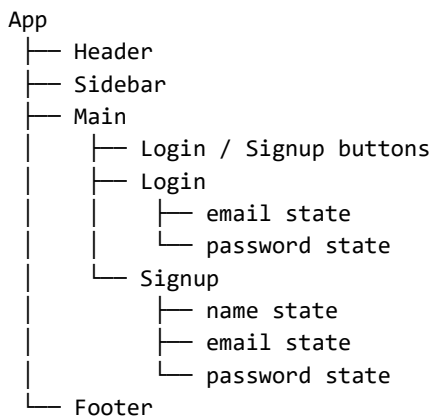
        {showLogin
          ? <Login title="Student Login" />
          : <Signup />
        }
      </main>

      <Footer />
    </div>
  );
}

export default App;

```

19. Final Application Flow



20. Props vs State

Props	State	Example
Passed from parent	Managed by component	title prop / email state
Used to pass data	Used for changing data	<Login title="Student Login" />
Read-only in child	Updated using setter	setEmail(...)

21. Important JSX Differences from HTML

HTML: <div class="page">
 JSX: <div className="page">

HTML: <label for="email">
 JSX: <label htmlFor="email">

22. Testing Checklist

- Run npm run dev and open the application in the browser.
- Check that Header, Sidebar, Main and Footer are displayed.
- Check that the Header title and user are received through props.
- Type an email and password into Login.

- Submit Login and check the browser console.
- Switch to Signup.
- Enter name, email and password.
- Submit Signup and check the browser console.
- Switch between Login and Signup using the buttons.

23. Common Errors to Check

- Use className instead of class in JSX.
- Check component file names and import paths.
- Make sure useState is imported from React.
- Make sure every component has export default ComponentName.
- Use value together with onChange when controlling an input with state.
- Use e.preventDefault() inside form submit handlers.
- Check the browser console for errors.

24. Practice Task

Extend the application by adding the following:

- Add a phone number field to Signup.
- Add a Confirm Password field.
- Display a simple message after successful form submission.
- Add a Logout button after Login.
- Pass a different title to Login using props.
- Create a reusable Button component and pass its text using props.

25. Quick Revision

Component	→ Builds part of the UI
JSX	→ HTML-like syntax in React
Props	→ Parent-to-child data
useState	→ Creates component state
setState	→ Updates state
onChange	→ Handles input changes
onSubmit	→ Handles form submission
preventDefault	→ Stops normal form reload
? :	→ Conditional rendering

End of Student Reference Notes